МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ

по лабораторной работе №1
по дисциплине «Построение и Анализ Алгоритмов»
Тема: Поиск с возвратом

Студент гр. 4342 Ларионов В.А.

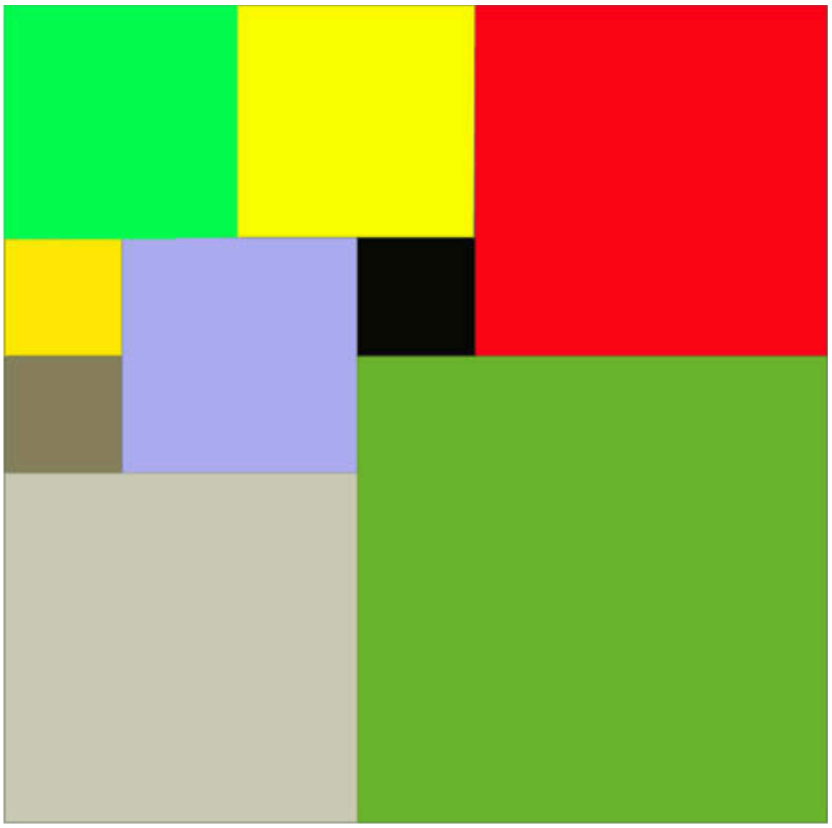
Преподаватель Фирсов М.А.

Санкт-Петербург
2026

Задание.

У Вовы много квадратных обрезков доски. Их стороны (размер) изменяются от 1 до $N - 1$, и у него есть неограниченное число обрезков любого размера. Но ему очень хочется получить большую столешницу - квадрат размера N . Он может получить ее, собрав из уже имеющихся обрезков(квадратов).

Например, столешница размера 7×7 может быть построена из 9 обрезков.



Внутри столешницы не должно быть пустот, обрезки не должны выходить за пределы столешницы и не должны перекрываться. Кроме того, Вова хочет использовать минимально возможное число обрезков.

Входные данные

Размер столешницы - одно целое число N $(2 \leq N \leq 20)$.

Выходные данные

Одно число K, задающее минимальное количество обрезков(квадратов), из которых можно построить столешницу(квадрат) заданного размера N. Далее должны идти K строк, каждая из которых должна содержать три целых числа

x, y и w, задающие координаты левого верхнего угла ($1 \leq x, y \leq N$) и длину стороны соответствующего обрезка(квадрата).

Пример входных данных

7

Соответствующие выходные данные

9

1 1 2

1 3 2

3 1 1

4 1 1

3 2 2

5 1 3

4 4 4

1 5 3

3 4 1

Индивидуализация для лаб. работы № 1:

Вар. 5р. Рекурсивный бэктрекинг. Возможность задать список квадратов (от 0 до N^2 квадратов в списке), которые обязательно должны быть использованы в покрытии квадрата со стороной N. Каждый квадрат из списка задаётся тремя числами, определяющими координаты и размер.

Описание алгоритма.

Алгоритм решает задачу разбиения квадрата размера $N \times N$ на минимальное количество непересекающихся квадратов с целыми сторонами с использованием рекурсивного поиска с возвратом и оптимизациями. Сетка представляется в виде битовой структуры: каждая строка хранится в 64-битном беззнаковом числе, где единичный бит означает занятую клетку, а нулевой — свободную. Поскольку по условию N не превышает 40, каждая строка полностью помещается в один *uint64_t*, что позволяет выполнять проверку

пересечений и размещение квадратов с помощью быстрых побитовых операций.

Основной алгоритм представляет собой полный перебор возможных разбиений с возвратом. На каждом рекурсивном вызове рассматривается частично заполненная сетка. Если текущее количество квадратов равно лучшему найденному решению, дальнейший перебор этой ветви прекращается, поскольку улучшить результат невозможно. Если сетка полностью заполнена, текущее решение сохраняется. В противном случае выбирается первая свободная клетка при просмотре строк сверху вниз и внутри строки слева направо. Это устраняет избыточный перебор симметричных конфигураций. Для выбранной клетки определяется максимальная возможная сторона квадрата, который можно разместить в этой позиции. Максимальная сторона ограничена границами сетки и уже занятыми клетками.

Проверка возможности размещения выполняется с помощью побитовых операций, для каждой строки предполагаемого квадрата формируется битовая маска, и проверяется отсутствие пересечения с уже установленными битами. Далее стороны перебираются от максимально возможной к единице. Такой порядок способствует более быстрому нахождению решений с малым числом крупных квадратов, что усиливает эффективность отсечения по текущему лучшему результату. После размещения квадрата выполняется рекурсивный вызов с обновленной сеткой, а затем состояние сетки восстанавливается, что обеспечивает корректную работу возврата.

Оптимизации алгоритма.

Перед запуском основного перебора применяется оптимизация, уменьшающая размер задачи. Если исходный размер N имеет простой делитель p, то задача сводится к квадрату размера $p \times p$. При этом вычисляется коэффициент масштабирования $scale = N / p$, далее поиск ведётся для меньшего квадрата, а полученное решение масштабируется обратно.

Дополнительно перед рекурсией используется предзаполнение известными конфигурациями для некоторых значений N. Если N кратно двум,

квадрат разбивается на четыре равных квадрата со стороной $N/2$. Если N кратно трём, используется известная конструкция из шести квадратов. Если N является простым числом, применяется стандартная конфигурация из трёх квадратов размеров $N/2 + 1$ в точке 0,0 и двух $N/2$ справа и снизу от него. Эти шаблоны позволяют либо полностью решить задачу без перебора, либо существенно уменьшить глубину дальнейшего поиска.

Кроме основных оптимизаций, описанных выше, во время разработки алгоритма были обнаружены некоторые малые оптимизации:

1) Ранее в *GridState* хранилась полностью заполненная сетка, теперь только строка, благодаря этому уменьшилось количество необходимой для работы алгоритма памяти и скорость проверки заполненности сетки;
2) Ранее *lineMask* вычислялась внутри place, после повторно в remove. Сейчас и place и remove используют одну и ту же *lineMask*, что сократило количество вычислений в ~2 раза;
3) Вместо сильного ветвления использовать флаги компиляции для компиляций нужных версий программы.

Сложность алгоритма.

Основной алгоритм является полным перебором с возвратом, он отсекает ветви рекурсии по лучшему решению, но по прежнему является экспоненциальным по сложности.

Вспомогательные функции *place remove canPlace getFirstPosToNextSquare* и *isGridFull* итерируются по строкам сетки, их сложность – $O(N)$.

По памяти в худшем случае $O(N\text{^}2)$ так как если нужно будет разбить сетку на единичные квадраты, то их будет необходимо записать в вектор текущего решения. Кроме этого, стоит учитывать сетку, количество памяти для нее линейно $O(N\text{^}2)$.

Описание функций и структур данных.

Структура *Position* предназначена для хранения координат точки в сетке. Она содержит поля:

- *size_t x* – координата по горизонтали, начало координат слева;

- *size_t y* – координата по вертикали, начало координат сверху.

Структура *Square* предназначена для хранения информации о квадрате, размещенном в сетке. Она содержит поля:

- *Position squareLeftUpCoordinates* – координаты левого верхнего угла;
- *size_t squareSide* – длина стороны квадрата.

Структура *GridState* предназначена для хранения текущего состояния сетки и всей вспомогательной информации, необходимой для рекурсивного поиска минимального разбиения. Она содержит поля:

- *size_t currentPartsCount* – количество квадратов, размещённых в текущей ветке рекурсии;
- *size_t bestPartsCount* – минимальное найденное количество квадратов среди всех рассмотренных ветвей;
- *size_t gridSide* – длина стороны сетки;
- *uint64_t* binGrid* – массив строк сетки, каждая строка хранится в виде 64-битной битовой маски, где единичные биты обозначают занятые клетки;
- *uint64_t fullLineMask* – маска полностью заполненной строки (первые gridSide бит равны единице);

(при COMPLEXITY_ANALYSIS = 1)

- *uint64_t calls* – количество рекурсивных вызовов функции поиска;

(при COMPLEXITY_ANALYSIS = 1)

- *uint64_t canPlaceChecks* – количество вызовов функции canPlace;
- *std::vector<Square> currentSolution* – список квадратов, размещённых в текущей ветке рекурсии;
- *std::vector<Square> bestSolution* – список квадратов, соответствующий лучшему найденному разбиению.

Функция *bool fitsInsideGrid(size_t gridSide, size_t squareSide, Position leftUpCoord)* – проверка выхода квадрата за границы сетки; *gridSide* – сторона сетки; *squareSide* – сторона проверяемого квадрата; *leftUpCoord* – координаты

левого верхнего угла квадрата; функция возвращает true, если квадрат полностью помещается в пределах сетки, и *false* в противном случае.

Функция *bool canPlace(GridState& state, size_t squareSide, Position leftUpCoord)* – проверка возможности размещения квадрата в текущем состоянии сетки; state – текущее состояние сетки; *squareSide* – сторона квадрата; *leftUpCoord* – координаты левого верхнего угла квадрата; функция возвращает *true*, если квадрат не выходит за границы сетки и не пересекается с уже размещенными квадратами, и *false* иначе.

Функция *uint64_t getLineMask(size_t squareSide, size_t xCoord)* – формирование битовой маски строки для размещаемого квадрата; *squareSide* – сторона квадрата; *xCoord* – координата по горизонтали, начиная с которой строится маска; функция возвращает 64-битное число, в котором подряд *squareSide* младших битов, начиная с позиции *xCoord*, установлены в единицу.

Функция *void place(GridState& state, uint64_t lineMask, size_t squareSide, size_t y)* – размещение квадрата в сетке; *state* – текущее состояние сетки; *lineMask* – битовая маска строки квадрата; *squareSide* – сторона квадрата; *y* – координата верхней строки квадрата; функция не возвращает значение и устанавливает соответствующие биты в строках сетки.

Функция *void remove(GridState& state, uint64_t lineMask, size_t squareSide, size_t y)* – удаление ранее размещенного квадрата из сетки; *state* – текущее состояние сетки; *lineMask* – битовая маска строки квадрата; *squareSide* – сторона квадрата; *y* – координата верхней строки квадрата; функция не возвращает значение и очищает (зануляет) соответствующие биты в строках сетки.

Функция Position *getFirstPosToNextSquare(GridState& state)* – поиск первой свободной клетки в сетке; *state* – текущее состояние сетки; функция возвращает координаты первой свободной клетки при просмотре строк сверху вниз и слева направо; если свободных клеток нет, возвращаются координаты (0, 0).

Функция *bool isGridFull(GridState& state)* – проверка полной заполненности сетки; *state* – текущее состояние сетки; функция возвращает true, если каждая строка сетки совпадает с полной маской *fullLineMask*, и *false* если найдена хотя бы одна незаполненная строка.

Функция *size_t getMaxSquareAtPosition(GridState& state, Position leftUpCoord)* – определение максимальной стороны квадрата, который можно разместить в заданной позиции; *state* – текущее состояние сетки; *leftUpCoord* – координаты левого верхнего угла предполагаемого квадрата; функция возвращает максимальное значение стороны квадрата, который можно разместить без выхода за границы и пересечений; если размещение невозможно, возвращается 0.

Функция *void findMinimalNumberOfPartsRecursive(GridState& state)* – рекурсивный перебор всех возможных вариантов разбиения методом возврата; *state* – текущее состояние сетки; функция не возвращает значение, но изменяет поля структуры *state*, обновляя *bestSolution* и *bestPartsCount* при нахождении более оптимального разбиения.

Функция *GridState getEmptyGridState(size_t correctSquareSide)* – создание пустого состояния сетки; *correctSquareSide* – сторона создаваемой сетки; функция возвращает структуру GridState с нулевым количеством размещенных квадратов, максимальным значением *bestPartsCount*, выделенной и обнулённой битовой сеткой и пустыми списками решений.

Функция *bool isPrime(size_t N)* – проверка числа на простоту; *N* – проверяемое число; функция возвращает *true*, если число является простым, и *false* в противном случае.

Функция *void prefillGridForKnownPatterns(GridState& state)* – предзаполнение сетки известными шаблонами оптимальных разбиений; *state* – текущее состояние сетки; функция не возвращает значение, но может изменить *binGrid*, *currentSolution* и *currentPartsCount* в зависимости от размера сетки.

Функция *size_t reduceToPrimeBase(size_t& N)* – сокращение размера сетки до простого делителя; *N* – сторона сетки, передаётся по ссылке и может быть

изменена; функция возвращает коэффициент масштабирования scale, равный отношению исходного *N* к найденному простому делителю; если сокращение невозможно, возвращается 1.

Функция *bool individualizationFunction(GridState& state)* – задание начального заполнения сетки пользователем (используется при STEPIK = 0); state – текущее состояние сетки; функция возвращает true при корректном вводе данных и успешном размещении всех заданных квадратов, и false при обнаружении ошибки входных данных.

Функция *void printStepikAnswer(GridState& state, size_t scale)* – вывод результата работы программы; *state* – текущее состояние сетки; *scale* – коэффициент масштабирования, примененный к исходному размеру; функция не возвращает значение и выводит минимальное количество квадратов и параметры каждого квадрата с учётом масштабирования.

Функция *void getOptimisation(GridState& state, size_t& N, size_t& scale)* – применение оптимизаций масштабирования и предзаполнения; *state* – текущее состояние сетки; *N* – сторона сетки, может быть изменена; *scale* – коэффициент масштабирования, может быть изменен; функция не возвращает значение и при необходимости пересоздает состояние сетки и выполняет предзаполнение.

Функция *int findTheMinimumPartition()* – основная управляющая функция программы; функция возвращает EXIT_SUCCESS при корректном завершении работы программы и EXIT_FAILURE при ошибке входных данных.

Функция *int main()* – точка входа в программу; функция возвращает результат выполнения функции *findTheMinimumPartition()*.

Тестирование.

Код unit-тестов приведен в Приложении А. Тестирование приведено в таблице 1 в Приложении Б.

Исследование.

Эта часть отчета была не обязательна для моего варианта, но мне было интересно провести исследование. На рис.1 представлен график зависимости

времени выполнения программы от размера квадрата N, временная шкала логарифмическая для наглядности.

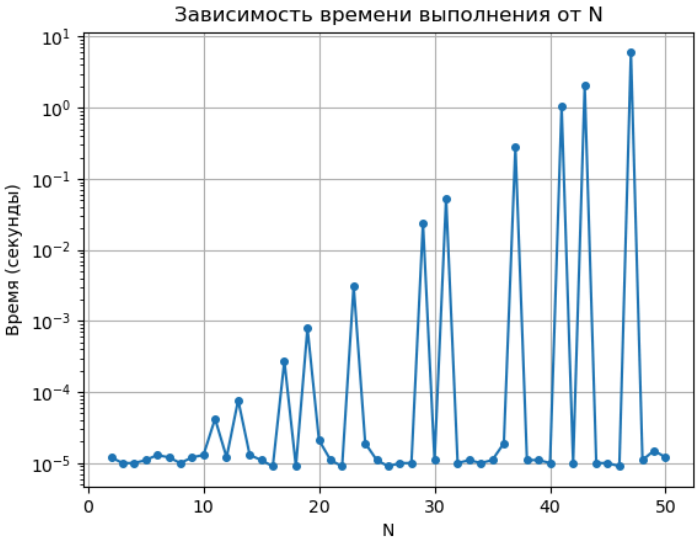


Рис.1 - график зависимости времени от размера квадрата

На графике можно наблюдать пики, которые возникают на местах вычисления программой квадрирования простого числа. Остальные числа вычисляются существенно быстрее, так для N кратных двум и трем скорость выполнения $O(1)$ и почти моментальная (порядка 0,0001 секунды), а для остальных N производится сокращение до простого числа, что уменьшает количество вычислений в разы.

Так как шкала времени логарифмическая, а простые числа (пики) образуют линейный график, можно утверждать об экспоненциальном росте сложности алгоритма по времени.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл main.cpp

```
#include <climits>
#include <iostream>
#include <vector>
#include <cstdint>
#include <algorithm>
/*
Если при компиляции добавить флаг STEPIK=0, то будет скопмилированна версия
предусмотренная индивидуализацией. При проверке на stepik.org по умолчанию будет
выставлено значение STEPIK=1 и код будет корректно выполен в проверяющей
системе.
*/
#ifndef STEPIK
    #define STEPIK 1
#endif
/*
Если при компиляции добавить флаг COMPLEXITY_ANALYSIS=1, то будет
скопмилированна версия для записи и вывода в консоль времени работы программы,
количетсва вызовов рекурсии и количества вызовов функции canPlace. По умолчанию
COMPLEXITY_ANALYSIS=0, что позволяет проходить проверку на stepik.org.
*/
#ifndef COMPLEXITY_ANALYSIS
    #define COMPLEXITY_ANALYSIS 0
#endif
/*
Если COMPLEXITY_ANALYSIS == 1, то добавляет нужные для вывода и записи
библиотеки.
*/
#if COMPLEXITY_ANALYSIS
    #include <cstdio>
    #include <ctime>
#endif
/*
Если при компиляции добавить флаг PRINT_INFO=1, то будет скомпилированна версия
программы, которая выводит больше информации для проверки работы или отладки.
*/
#ifndef PRINT_INFO
    #define PRINT_INFO 0
#endif
/*
Если PRINT_INFO == 1, то добавляет нужные для вывода битового представления
строк матрицы.
*/
#if PRINT_INFO
    #include <bitset>
    #include <string>
    #include <fstream>
    std::ofstream out("intermediate_results.txt");
#endif


/*
Структура для хранения информации о координатах.
*/
```

```
struct Position {
    size_t x;
    size_t y;
};


/*
Структура для хранения инфомарции о квадрате.
- Position squareLeftUpCoordinates -- позиция левого верхнего угла квадрата
- size_t squareSide -- сторона квадрата
*/
struct Square {
    Position squareLeftUpCoordinates;
    size_t squareSide;
};


/*
Структура для хранения и передачи информации о квадрате, который планируем
квадрировать (далее - сетка) и вспомогательной информации через рекурсивые
вызовы.
- size_t currentPartsCount -- количество размещенных в сетке квадратов в текущей
ветке рекурсии.
- size_t bestPartsCount -- лучшее среди всех ветвей количество размещенных
в сетке квадратов.
- size_t gridSide -- сторона сетки.
- uint64_t* binGrid -- массив битового представления строк сетки.
- uint64_t fullLineMask -- маска для строки, чтобы не вычислять каждый раз.
- std::vector<Square> currentSolution -- вектор для хранения координат
и сторон квадратов размещенных в сетке в текущей ветке рекурсии.
- std::vector<Square> bestSolution -- вектор для хранения лучших координат и
сторон квадратов размещенных в сетке среди всех веток рекурсии.
Если при компиляции был задан флаг COMPLEXITY_ANALYSIS=1, то в структуру
добавляются еще два поля, которые нужны для определения сложности алгоритма:
- uint64_t calls -- количество рекурсивных вызовов. Увеличивается при каждом
вызове findMinimalNumberOfPartsRecursive().
- uint64_t canPlaceChecks -- количество вызовов функции canPlace.
*/
struct GridState {
    size_t currentPartsCount;
    size_t bestPartsCount;
    size_t gridSide;
    uint64_t* binGrid;
    uint64_t fullLineMask;
    #if COMPLEXITY_ANALYSIS
        uint64_t calls;
        uint64_t canPlaceChecks;
    #endif
    std::vector<Square> currentSolution;
    std::vector<Square> bestSolution;
};


/*
Функция принимает на вход размер сетки gridSide, размер вставляемого квадрата
squareSide и координаты верхнего левого угла этого квадрата leftUpCoord.
Возвращает true, если квадрат не выходит за границу сетки и false, если выходит.
*/
bool fitsInsideGrid(size_t gridSide, size_t squareSide, Position leftUpCoord) {
    return leftUpCoord.x + squareSide <= gridSide &&
    leftUpCoord.y + squareSide <= gridSide;
}
```

```
/*
Функция принимает на вход ссылку на структуру GridState state,
размер вставляемого квадрата squareSide и координаты верхнего левого угла этого
квадрата leftUpCoord.
Возвращает true, если квадрат со стороной squareSide и координатами  leftUpCoord
можно вставить в сетку.
*/
bool canPlace(GridState& state, size_t squareSide, Position leftUpCoord) {
    #if COMPLEXITY_ANALYSIS
        ++state.canPlaceChecks;
    #endif
    if (!fitsInsideGrid(state.gridSide, squareSide, leftUpCoord)) {
        return false;
    }
    uint64_t line = ((1ULL << squareSide) - 1) << leftUpCoord.x;
    for (size_t i = 0; i < squareSide; ++i) {
        if (state.binGrid[leftUpCoord.y + i] & line) {
            return false;
        }
    }
    return true;
}

/*
Функция принимает на вход размер вставляемого квадрата squareSide и координату.
Возвращает маску для строки, соответствующую вставляемому квадрату.
*/
inline uint64_t getLineMask(size_t squareSide, size_t xCoord) {
    return ((1ULL << squareSide) - 1) << xCoord;
}


/*
Функция принимает на вход ссылку на структуру GridState state, маску для строк
lineMask, размер вставляемого квадрата squareSide и координату верхнего левого
угла этого квадрата yCoord.
Итерируясь от верхней координаты квадрата к нижней, вставляет максу строки в
строку сетки с помощью побитового ИЛИ с присваиванием.
Пример:
state.binGrid | squareSide | leftUpCoord |   line    | result  |
 0b00111      | 2          | {3,0}       |  0b11000  | 0b11111 |
 0b00111      |            |             |           | 0b11111 |
 0b00111      |            |             |           | 0b00111 |
 0b00000      |            |             |           | 0b00000 |
 0b00000      |            |             |           | 0b00000 |
*/
void place(GridState& state, uint64_t lineMask, size_t squareSide, size_t y) {
    for (size_t i = 0; i < squareSide; ++i) {
        state.binGrid[y + i] |= lineMask;
    }
}


/*
Функция принимает на вход ссылку на структуру GridState state, маску для строк
lineMask, размер вставляемого квадрата squareSide и координату верхнего левого
угла этого квадрата yCoord.
Итерируясь от верхней координаты квадрата к нижней, вставляет инвертированную
максу строки в строку сетки с помощью побитового И с присваиванием.
Пример:
state.binGrid | squareSide | leftUpCoord |   line    | result  |
 0b11111      | 2          | {3,0}       |  0b11000  | 0b00111 |
```

```
 0b11111      |            |             |           | 0b00111 |
 0b00111      |            |             |           | 0b00111 |
 0b00000      |            |             |           | 0b00000 |
 0b00000      |            |             |           | 0b00000 |
*/
void remove(GridState& state, uint64_t lineMask, size_t squareSide, size_t y) {
    for (size_t i = 0; i < squareSide; ++i) {
        state.binGrid[y + i] &= ~lineMask;
    }
}


/*
Функция получает на вход ссылку на структуру GridState, возвращает первую
левую сверху позицию.
Встроенная функция компилятора gcc (g++)__builtin_ctzll используется для
получения количества нулей в бинарном представлении строки стоящих в
младших битах.

Примеры:
 state.binGrid | free       | __builtin_ctzll | result  |
 0b00111       | 0b11000    | 3               | 3,0     |
 0b00111       |            |                 |         |
 0b00111       |            |                 |         |
 0b00000       |            |                 |         |
 0b00000       |            |                 |         |

 state.binGrid | free       | __builtin_ctzll | result  |
 0b1111111     | 0b0000000  | -               | -       |
 0b1111111     | 0b0000000  | -               | -       |
 0b1111111     | 0b0000000  | -               | -       |
 0b1111111     | 0b0000000  | -               | -       |
 0b0000011     | 0b1111100  | 2               | 2,5     |
 0b0000000     |            |                 |         |
 0b0000000     |            |                 |         |
*/
Position getFirstPosToNextSquare(GridState& state) {
    for (size_t y = 0; y < state.gridSide; ++y) {
        uint64_t free = ~state.binGrid[y] & state.fullLineMask;
        if (free) {
            size_t x = __builtin_ctzll(free);
            return {x, y};
        }
    }
    return {0, 0};
}

/*
Функция получает на вход ссылку на структуру GridState, возвращает false, если
находит не до конца заполненную строку и true, если прошла по всем строкам и
не нашла строк не равных полной маске строки.
*/
bool isGridFull(GridState& state) {
    for (size_t i = 0; i < state.gridSide; ++i) {
        if ((state.binGrid[i] & state.fullLineMask) != state.fullLineMask) {
            return false;
        }
    }
    return true;
}


/*
```

```
Функция получает на вход ссылку на структуру GridState и координаты левого
верхнего угла квадрата leftUpCoord. Возвращает максимальную сторону квадрата,
который можно поместить в leftUpCoord.
*/
size_t getMaxSquareAtPosition(GridState& state, Position leftUpCoord) {
    size_t dx = state.gridSide - leftUpCoord.x;
    size_t dy = state.gridSide - leftUpCoord.y;
    size_t maxPossibleSquareSide = std::min(dx, dy);
    if (maxPossibleSquareSide > state.gridSide - 1) {
        maxPossibleSquareSide = state.gridSide - 1;
    }
    for (size_t side = maxPossibleSquareSide; side > 0; --side) {
        if (canPlace(state, side, leftUpCoord)) {
            return side;
        }
    }
    return 0;
}


#if PRINT_INFO
/*
Фунция принимает на вход ссылку на структуру GridState. Выводит в консоль
сетку переворачивая и обрезая битовое представление.
*/
void printGrid(GridState& state) {
    out << '\n';
    for (size_t y = 0; y < state.gridSide; ++y) {
        std::string strBinLine = std::bitset<64>(state.binGrid[y]).to_string();
        std::reverse(strBinLine.begin(), strBinLine.end());
        out << strBinLine.substr(0, state.gridSide) << '\n';
    }
    out << '\n';
}

void printFilledGrid(GridState& state, std::vector<Square> solution) {
    out << "\n";
    size_t N = state.gridSide;
    std::vector<std::vector<size_t>> grid(N, std::vector<size_t>(N, 0));
    for (size_t idx = 0; idx < solution.size(); ++idx) {
        size_t squareNumber = idx + 1;
        size_t x = solution[idx].squareLeftUpCoordinates.x - 1;
        size_t y = solution[idx].squareLeftUpCoordinates.y - 1;
        size_t side = solution[idx].squareSide;
        for (size_t dy = 0; dy < side; ++dy) {
            for (size_t dx = 0; dx < side; ++dx) {
                grid[y + dy][x + dx] = squareNumber;
            }
        }
    }
    for (size_t i = 0; i < N; ++i) {
        for (size_t j = 0; j < N; ++j) {
            out << grid[i][j] << " ";
        }
        out << "\n";
    }
    out << "\n";
}
#endif


/*
Функция получает на вход ссылку на структуру GridState state.
```

```
Рекурсивно перебирает все возможные варианты разбиения сетки на квадраты
с использованием возврата (backtracking).

Алгоритм работы:
1) Если текущее количество квадратов в ветке >= лучшему найденному,
   дальнейший перебор не имеет смысла — происходит отсечение ветки.
2) Если сетка полностью заполнена, обновляется лучшее решение.
3) Находится первая свободная сверху слева позиция.
4) В этой позиции перебираются все возможные стороны квадрата
   (от максимальной к минимальной).
5) Квадрат размещается, вызывается рекурсия, затем происходит откат состояния.

Если при компиляции задан флаг COMPLEXITY_ANALYSIS=1,
то дополнительно увеличивается счетчик рекурсивных вызовов.
*/
void findMinimalNumberOfPartsRecursive(GridState& state) {
    #if COMPLEXITY_ANALYSIS
        ++state.calls;
    #endif
    if (state.currentPartsCount == state.bestPartsCount) {
        #if PRINT_INFO
            out << "Текущее разбиение currentPartsCount = "
                << state.currentPartsCount
                << " квадратов,\nчто равно лучшему разбиению bestPartsCount = "
                << state.bestPartsCount << ".\nОбрезаем эту ветвь рекурсии.\n";
            printFilledGrid(state, state.currentSolution);
        #endif
        return;
    }
    if (isGridFull(state)) {
        #if PRINT_INFO
            out << "Найдено полное разбиение.\nПредыдущее лучшее разбиение: "
                                << (state.bestPartsCount == SIZE_MAX ? "-" :
std::to_string(state.bestPartsCount))
                << "\nНовое лучшее разбиение: "
                << state.currentPartsCount << "\n";
            printFilledGrid(state, state.currentSolution);
        #endif
        state.bestSolution = state.currentSolution;
        state.bestPartsCount = state.currentPartsCount;
        return;
    }
    Position leftUpCoord = getFirstPosToNextSquare(state);
    size_t maxSide = getMaxSquareAtPosition(state, leftUpCoord);
    for (size_t side = maxSide; side > 0; --side) {
        uint64_t currentLineMask = getLineMask(side, leftUpCoord.x);
        #if PRINT_INFO
            out << "Квадрат со стороной " << side
                << " поставлен в координаты x = " << leftUpCoord.x
                << " y = " << leftUpCoord.y << "\n";
        #endif
        place(state, currentLineMask, side, leftUpCoord.y);
        Square currentSquare = {{leftUpCoord.x + 1, leftUpCoord.y + 1}, side};
        state.currentSolution.push_back(currentSquare);
        ++state.currentPartsCount;
        findMinimalNumberOfPartsRecursive(state);
        #if PRINT_INFO
            out << "Квадрат со стороной " << side
                << " удален по координатам x = " << leftUpCoord.x
                << " y = " << leftUpCoord.y << "\n";
        #endif
        remove(state, currentLineMask, side, leftUpCoord.y);
        state.currentSolution.pop_back();
```

```
        --state.currentPartsCount;
    }
}


/*
Функция принимает на вход размер сетки correctSquareSide.
Создает пустую структуру GridState:
- выделяет динамический массив строк binGrid,
- инициализирует его нулями,
- вычисляет полную маску строки fullLine,
- устанавливает currentPartsCount = 0,
- устанавливает bestPartsCount = INT_MAX,
- инициализирует векторы решений как пустые.

Возвращает созданную структуру GridState.
*/
GridState getEmptyGridState(size_t correctSquareSide) {
    uint64_t* emptyGrid = new uint64_t[correctSquareSide]{};
    uint64_t fullLine = (1ULL << correctSquareSide) - 1;
    return {
        0,
        SIZE_MAX,
        correctSquareSide,
        emptyGrid,
        fullLine,
        #if COMPLEXITY_ANALYSIS
            0,
            0,
        #endif
        {},
        {}
    };
}


/*
Функция принимает на вход число N.
Возвращает true, если N является простым числом, иначе false.

Используется оптимизированная проверка:
- сначала отсекаются N <= 3,
- затем проверка делимости на 2 и 3,
- далее перебор делителей вида 6k ± 1 до sqrt(N).
*/
bool isPrime(size_t N) {
    if (N <= 1) return false;
    if (N == 2 || N == 3) return true;
    if (N % 2 == 0 || N % 3 == 0) return false;
    for (size_t i = 5; i * i <= N; i += 6) {
        if (N % i == 0 || N % (i + 2) == 0) {
            return false;
        }
    }
    return true;
}


/*
Функция принимает на вход ссылку на структуру GridState state.
В зависимости от размера сетки N выполняет предзаполнение сетки
известными шаблонами разбиений, что позволяет сократить перебор.
```

```
1) Если N кратно 2:
   Сетка разбивается на 4 равных квадрата со стороной N/2.

2) Если N кратно 3:
   Используется известное разбиение на 6 квадратов:
   один большой 2k x 2k и пять квадратов k x k.

3) Если N — простое:
   Используется стандартная конфигурация из трех квадратов:
   - один большой (N/2 + 1),
   - два квадрата (N/2).

Во всех случаях:
- квадраты размещаются в binGrid,
- добавляются в currentSolution,
- увеличивается currentPartsCount.
*/
void prefillGridForKnownPatterns(GridState& state) {
    size_t N = state.gridSide;
    if (N % 2 == 0) {
        #if PRINT_INFO
            out << "Сторона квадрата " << N << " делится на два.\n"
                << "Лучшее разбиение 4."
                << " Рекурсия не требуется, задача решена.";
        #endif
        size_t half = N / 2;
        size_t coords[2] = {0, half};
        for (size_t y0 : coords) {
            for (size_t x0 : coords) {
                place(state, getLineMask(half, x0), half, y0);
                state.currentSolution.push_back({{x0 + 1, y0 + 1}, half});
                ++state.currentPartsCount;
            }
        }
    }
    else if (N % 3 == 0) {
        #if PRINT_INFO
            out << "Сторона квадрата " << N << " делится на три.\n"
                << "Лучшее разбиение 6."
                << " Рекурсия не требуется, задача решена.";
        #endif
        size_t k = N / 3;
        size_t k2 = 2 * k;
        size_t X[6] = {0, k2, k2, 0, k, k2};
        size_t Y[6] = {0, 0, k, k2, k2, k2};
        size_t S[6] = {k2, k, k, k, k, k};
        for (int i = 0; i < 6; ++i) {
            place(state, getLineMask(S[i], X[i]), S[i], Y[i]);
            state.currentSolution.push_back({{X[i] + 1, Y[i] + 1}, S[i]});
            ++state.currentPartsCount;
        }
    }
    else if (isPrime(N)) {
        #if PRINT_INFO
            out << "Сторона квадрата " << N << " -- простое число.\n"
                << "Можно использовать оптимизацию, в лучшем результате всегда"
                 << " будет 3 квадрата: \nодин со стороной N / 2 + 1 в координатах
0,0 "
                << "\nи два со сторонами N / 2 в координатах (N / 2 + 1,0) и (0, N
/ 2 + 1)\n";
        #endif
        size_t s1 = N / 2 + 1;
        size_t s2 = N / 2;
```

```
        place(state, getLineMask(s1, 0), s1, 0);
        state.currentSolution.push_back({{1, 1}, s1});
        ++state.currentPartsCount;
        place(state, getLineMask(s2, 0), s2, s2 + 1);
        state.currentSolution.push_back({{1, s2 + 2}, s2});
        ++state.currentPartsCount;
        place(state, getLineMask(s2, s2 + 1), s2, 0);
        state.currentSolution.push_back({{s2 + 2, 1}, s2});
        ++state.currentPartsCount;
    }
}


/*
Функция принимает на вход ссылку на размер сетки N.
Пытается сократить размер сетки до простого делителя.

Если находится простой делитель p:
- вычисляется scale = N / p,
- N заменяется на p,
- возвращается scale.

Если делитель не найден,
возвращается 1 (масштабирование не требуется).
*/
size_t reduceToPrimeBase(size_t& N) {
    for (size_t p = 2; p * p <= N; ++p) {
        if (N % p == 0 && isPrime(p)) {
            size_t scale = N / p;
            N = p;
            return scale;
        }
    }
    return 1;
}


#if !STEPIK
/*
Функция принимает на вход ссылку на структуру GridState state.
Используется только при компиляции с флагом STEPIK=0.

Позволяет задать начальное состояние сетки:
1) Считывается количество квадратов numberOfSquares.
2) Для каждого квадрата считываются:
   - координаты левого верхнего угла (x, y) в 1-индексации,
   - сторона квадрата.
3) Проверяется:
   - корректность координат (>= 1),
   - корректность стороны квадрата,
   - размещение квадрата внутри сетки,
   - отсутствие пересечения с уже размещёнными квадратами.
4) Координаты переводятся во внутреннюю 0-индексацию.
5) Квадрат размещается в битовой сетке с помощью place().
6) Информация о квадрате сохраняется в currentSolution.
7) Увеличивается счётчик currentPartsCount.

Возвращает:
- true — если ввод корректен,
- false — если обнаружена ошибка входных данных.
*/
bool individualizationFunction(GridState& state) {
    int numberOfSquares = 0;
```

```
    std::cin >> numberOfSquares;

    Position leftUpCoord;
    long long currentSquareSide;

    for (int i = 0; i < numberOfSquares; ++i) {
        std::cin >> leftUpCoord.x >> leftUpCoord.y >> currentSquareSide;
        if (leftUpCoord.x < 1 || leftUpCoord.y < 1) {
            return false;
        }
                    if  (currentSquareSide  <  1  ||  currentSquareSide  >  (long
long)state.gridSide) {
            return false;
        }
        size_t xCoord = leftUpCoord.x - 1;
        size_t yCoord = leftUpCoord.y - 1;
           if (!fitsInsideGrid(state.gridSide, (size_t)currentSquareSide, {xCoord,
yCoord})) {
            return false;
        }
        if (!canPlace(state, (size_t)currentSquareSide, {xCoord, yCoord})) {
            return false;
        }
        uint64_t lineMask = getLineMask((size_t)currentSquareSide, xCoord);
        place(state, lineMask, (size_t)currentSquareSide, yCoord);
                      Square  currentSquare  =  {{xCoord  +  1,  yCoord  +  1},
(size_t)currentSquareSide};
        state.currentSolution.push_back(currentSquare);
        state.currentPartsCount++;
    }
    return true;
}
#endif


/*
Функция принимает на вход ссылку на структуру GridState state
и коэффициент масштабирования scale.

Выводит:
1) минимальное количество квадратов,
2) координаты и стороны каждого квадрата.

Если применялось масштабирование (scale > 1),
то координаты и стороны квадратов домножаются на scale,
при этом координата 1 остается неизменной.
*/
void printStepikAnswer(GridState& state, size_t scale) {
    std::cout << state.bestPartsCount << "\n";
    for (const auto& sq : state.bestSolution) {
        size_t x = sq.squareLeftUpCoordinates.x;
        size_t y = sq.squareLeftUpCoordinates.y;
        size_t s = sq.squareSide;
        if (scale > 1) {
            if (x != 1) x = (x - 1) * scale + 1;
            if (y != 1) y = (y - 1) * scale + 1;
            s *= scale;
        }
        std::cout << x << " " << y << " " << s << "\n";
    }
}
```

```
/*
Функция принимает на вход ссылку на структуру GridState state, ссылку на размер
сетки N и ссылку на показатель масштаба scale. Ничего не возвращает, но может
изменять state, N и scale в зависимости от значений N.
Если количество размещенных в сетке квадратов = 0, то применяется оптимизация
сокращения стороны сетки до простого числа, в ином случае, работа функции
заканчивается.
Если оптимизация сокращения стороны применяется, то создается новая пустая
сетка, меньше предыдущей на scale.
После, если scale был изменен (>1), создается новая сетка state, меньше
предыдущей на scale, далее сетка передается в функцию
void prefillGridForKnownPatterns(GridState& state) для предзаполнения, если
это возможно.
*/
void getOptimisation(GridState& state, size_t& N, size_t& scale) {
    if (state.currentPartsCount == 0) {
        scale = reduceToPrimeBase(N);
        #if PRINT_INFO
            out << "Запущена функция getOptimisation().\n";
        #endif
        if (scale > 1) {
            #if PRINT_INFO
                 out << "Удалось сократить N = " << N*scale << " до простого числа
"
                    << N << ".\n";
            #endif
            state = getEmptyGridState(N);
        }
        prefillGridForKnownPatterns(state);
    }
}


/*
Функция является основной управляющей функцией программы.

Алгоритм работы:
1) Считывает размер исходного квадрата.
2) Проверяет корректность входных данных (2 <= N <= 63).
3) Создает пустую сетку.
4) При необходимости выполняет индивидуализацию.
5) Применяет оптимизации (масштабирование и предзаполнение).
6) Запускает рекурсивный поиск минимального разбиения.
7) Выводит ответ.
8) При COMPLEXITY_ANALYSIS=1 выводит статистику.
9) Освобождает динамически выделенную память.

Возвращает EXIT_SUCCESS или EXIT_FAILURE.
*/
int findTheMinimumPartition() {
    long long squareSide;
    std::cin >> squareSide;
    if (squareSide < 2 || squareSide > 63) {
        return EXIT_FAILURE;
    }
    size_t N = static_cast<size_t>(squareSide);
    GridState state = getEmptyGridState(N);
    size_t scale = 1;

    #if COMPLEXITY_ANALYSIS
        clock_t start = clock();
    #endif
```

```
    #if !STEPIK
        if (!individualizationFunction(state)) {
            return EXIT_FAILURE;
        }
    #endif

    getOptimisation(state, N, scale);
    findMinimalNumberOfPartsRecursive(state);
    printStepikAnswer(state, scale);
    #if PRINT_INFO
        out << "\nИтоговое разбиение:\n";
        printFilledGrid(state, state.bestSolution);
    #endif
    #if COMPLEXITY_ANALYSIS
        clock_t end = clock();
        double seconds = (double)(end - start) / CLOCKS_PER_SEC;
        printf("Время выполнения: %.6f секунд\n", seconds);
        printf("Рекурсивных вызовов: %lu\n", state.calls);
        printf("Проверок canPlace: %lu\n", state.canPlaceChecks);
    #endif

    delete[] state.binGrid;
    return EXIT_SUCCESS;
}


int main() {
    return findTheMinimumPartition();
}
```

Файл tests.cpp

```
#include <cassert>
#include <iostream>

#include "./squaring.hpp" // вынес все нужные функции в заголовочный файл squaring.hpp

void test_isPrime() {
    assert(!isPrime(0));
    assert(!isPrime(1));
    assert(isPrime(2));
    assert(isPrime(3));
    assert(!isPrime(4));
    assert(isPrime(5));
    assert(!isPrime(9));
    assert(isPrime(61));
}

void test_fitsInsideGrid() {
    assert(fitsInsideGrid(5, 2, {0,0}));
    assert(fitsInsideGrid(5, 5, {0,0}));
    assert(!fitsInsideGrid(5, 3, {3,3}));
    assert(!fitsInsideGrid(5, 6, {0,0}));
}

void test_place_and_remove() {
    GridState state = getEmptyGridState(4);

    uint64_t mask = getLineMask(2, 1);
    place(state, mask, 2, 0);

    assert(state.binGrid[0] == 0b0110);
    assert(state.binGrid[1] == 0b0110);
```

```
    remove(state, mask, 2, 0);

    assert(state.binGrid[0] == 0);
    assert(state.binGrid[1] == 0);

    delete[] state.binGrid;
}

void test_canPlace() {
    GridState state = getEmptyGridState(4);

    assert(canPlace(state, 2, {0,0}));
    place(state, getLineMask(2,0), 2, 0);

    assert(!canPlace(state, 2, {0,0}));
    assert(canPlace(state, 2, {2,0}));

    delete[] state.binGrid;
}

void test_reduceToPrimeBase() {
    size_t N = 25;
    size_t scale = reduceToPrimeBase(N);
    assert(scale == 5);
    assert(N == 5);

    N = 7;
    scale = reduceToPrimeBase(N);
    assert(scale == 1);
    assert(N == 7);
}

void test_prefill_even() {
    GridState state = getEmptyGridState(4);
    prefillGridForKnownPatterns(state);

    assert(state.currentPartsCount == 4);
    assert(isGridFull(state));

    delete[] state.binGrid;
}

void test_prefill_div3() {
    GridState state = getEmptyGridState(9);
    prefillGridForKnownPatterns(state);

    assert(state.currentPartsCount == 6);
    assert(isGridFull(state));

    delete[] state.binGrid;
}

int main() {
    test_isPrime();
    test_fitsInsideGrid();
    test_place_and_remove();
    test_canPlace();
    test_reduceToPrimeBase();
    test_prefill_even();
    test_prefill_div3();

    std::cout << "All tests passed!\n";
```

```
}
```

ПРИЛОЖЕНИЕ Б

ТЕСТИРОВАНИЕ

Таблица тестирования основной задачи:

Входные данные (сторона квадрата)	Выходные данные(число разбиения)	Комментарии
2	4 1 1 1 2 1 1 1 2 1 2 2 1	Корректное разбиение
3	6 1 1 2 3 1 1 3 2 1 1 3 1 2 3 1 3 3 1	Корректное разбиение
4	4 1 1 2 3 1 2 1 3 2 3 3 2	Корректное разбиение
5	8 1 1 3 1 4 2 4 1 2 4 3 2 3 4 1 3 5 1 4 5 1 5 5 1	Корректное разбиение
6	4 1 1 3 4 1 3 1 4 3 4 4 3	Корректное разбиение
7	9 1 1 4 1 5 3 5 1 3	Корректное разбиение

	5 4 2 7 4 1 4 5 1 7 5 1 4 6 2 6 6 2	
8	4 1 1 4 5 1 4 1 5 4 5 5 4	Корректное разбиение
9	6 1 1 6 7 1 3 7 4 3 1 7 3 4 7 3 7 7 3	Корректное разбиение
10	4 1 1 5 6 1 5 1 6 5 6 6 5	Корректное разбиение
11	11 1 1 6 1 7 5 7 1 5 7 6 3 10 6 2 6 7 1 6 8 1 10 8 1 11 8 1 6 9 3 9 9 3	Корректное разбиение
12	4 1 1 6 7 1 6 1 7 6 7 7 6	Корректное разбиение
13	11 1 1 7 1 8 6 8 1 6	Корректное разбиение

	8 7 2 10 7 4 7 8 1 7 9 3 10 11 1 11 11 3 7 12 2 9 12 2	
14	4 1 1 7 8 1 7 1 8 7 8 8 7	Корректное разбиение
15	6 1 1 10 11 1 5 11 6 5 1 11 5 6 11 5 11 11 5	Корректное разбиение
16	4 1 1 8 9 1 8 1 9 8 9 9 8	Корректное разбиение
17	12 1 1 9 1 10 8 10 1 8 10 9 2 12 9 4 16 9 2 9 10 1 9 11 3 16 11 2 12 13 1 13 13 5 9 14 4	Корректное разбиение
18	4 1 1 9 10 1 9 1 10 9 10 10 9	Корректное разбиение

19	13 1 1 10 1 11 9 11 1 9 11 10 3 14 10 6 10 11 1 10 12 1 10 13 4 14 16 1 15 16 1 16 16 4 10 17 3 13 17 3	Корректное разбиение
20	4 1 1 10 11 1 10 1 11 10 11 11 10	Корректное разбиение
21	6 1 1 14 15 1 7 15 8 7 1 15 7 8 15 7 15 15 7	Корректное разбиение
22	4 1 1 11 12 1 11 1 12 11 12 12 11	Корректное разбиение
23	13 1 1 12 1 13 11 13 1 11 13 12 2 15 12 5 20 12 4 12 13 1 12 14 3 20 16 1 21 16 3 12 17 7 19 17 2	Корректное разбиение

	19 19 5	
24	24 4 1 1 12 13 1 12 1 13 12 13 13 12	Корректное разбиение
25	8 1 1 15 1 16 10 16 1 10 16 11 10 11 16 5 11 21 5 16 21 5 21 21 5	Корректное разбиение
26	4 1 1 13 14 1 13 1 14 13 14 14 13	Корректное разбиение
27	6 1 1 18 19 1 9 19 10 9 1 19 9 10 19 9 19 19 9	Корректное разбиение
28	4 1 1 14 15 1 14 1 15 14 15 15 14	Корректное разбиение
29	14 1 1 15 1 16 14 16 1 14 16 15 2 18 15 5 23 15 7 15 16 1 15 17 3 15 20 3 18 20 3	Корректное разбиение

	21 20 2 21 22 1 22 22 8 15 23 7	
30	4 1 1 15 16 1 15 1 16 15 16 16 15	Корректное разбиение
31	15 1 1 16 1 17 15 17 1 15 17 16 3 20 16 6 26 16 6 16 17 1 16 18 1 16 19 4 20 22 1 21 22 1 22 22 10 16 23 6 16 29 3 19 29 3	Корректное разбиение
32	4 1 1 16 17 1 16 1 17 16 17 17 16	Корректное разбиение
33	6 1 1 22 23 1 11 23 12 11 1 23 11 12 23 11 23 23 11	Корректное разбиение
34	4 1 1 17 18 1 17 1 18 17 18 18 17	Корректное разбиение
35	8 1 1 21	Корректное разбиение

	1 22 14 22 1 14 22 15 14 15 22 7 15 29 7 22 29 7 29 29 7	
36	4 1 1 18 19 1 18 1 19 18 19 19 18	Корректное разбиение
37	15 1 1 19 1 20 18 20 1 18 20 19 2 22 19 5 27 19 11 19 20 1 19 21 3 19 24 8 27 30 3 30 30 8 19 32 6 25 32 1 26 32 1 25 33 5	Корректное разбиение
38	4 1 1 19 20 1 19 1 20 19 20 20 19	Корректное разбиение
39	6 1 1 26 27 1 13 27 14 13 1 27 13 14 27 13 27 27 13	Корректное разбиение
40	4 1 1 20 21 1 20 1 21 20	Корректное разбиение

	21 21 20	

Таблица 1 – Тестирование основного алгоритма.

Таблица тестирования индивидуализации:

Входные данные (сторона квадрата)	Входные данные (координаты и сторона квадратов)	Вывод алгоритма	Комментарии
7	2 1 1 3 4 4 1	13 1 1 3 4 4 1 4 1 3 7 1 1 7 2 1 7 3 1 1 4 2 3 4 1 5 4 1 6 4 2 3 5 3 1 6 2 6 6 2	Корректное разбиение
10	1 5 5 5	19 5 5 5 1 1 4 5 1 4 9 1 2 9 3 2 1 5 4 10 5 1 10 6 1 10 7 1 10 8 1 1 9 2 3 9 2 10 9 1 5 10 1 6 10 1 7 10 1 8 10 1 9 10 1	Корректное разбиение

		10 10 1	
14	3 1 1 6 7 10 3 11 12 1	19 1 1 6 7 10 3 11 12 1 7 1 6 13 1 2 13 3 2 13 5 2 1 7 6 7 7 3 10 7 5 10 12 1 12 12 3 1 13 2 3 13 2 5 13 2 7 13 2 9 13 2 11 13 1 11 14 1	Корректное разбиение

Таблица 2 – Тестирование индивидуализации.